

SQA



Đảm bảo chất lượng phần mềm

Software Quality Assurance



5. Kỹ thuật Kiểm thử

Nguyễn Anh Hào

Khoa CNTT2, Học viện Công Nghệ BCVT Tp.HCM

✉: nahao@ptithcm.edu.vn

☎: 0913609730

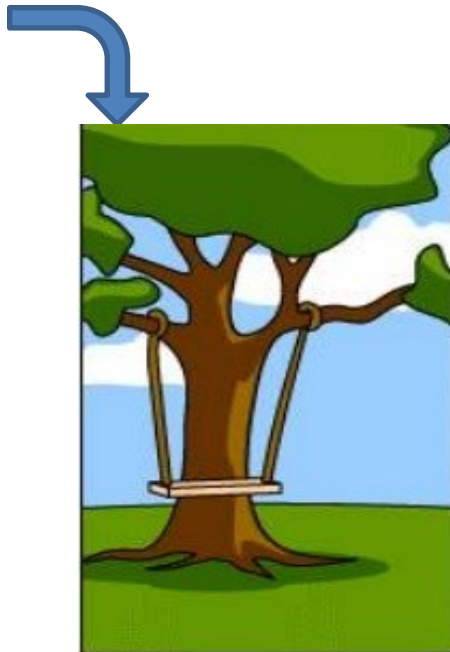
Tài liệu môn học – 2025

Lỗi của phần mềm

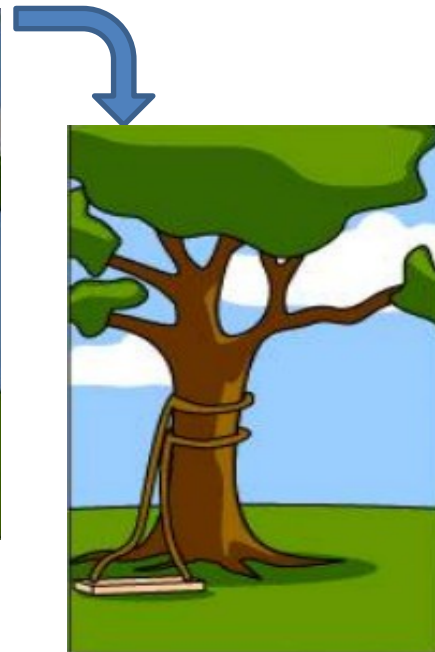
3



ANALISYS



DESIGN



CODE



DEBUG

Lỗi phần mềm : ngôn từ

4

- ❖ **Error**: là “sự hư hỏng” trong bản thân phần mềm (vd: logic bị sai) theo quan điểm của developers.
- ❖ **Fault**: là “sự hư hỏng” trong cách xử lý của phần mềm do error gây ra khi phần mềm đang chạy.
- ❖ **Failure**: là “sự hư hỏng” bộc lộ ra ngoài của fault khi phần mềm đang chạy đụng đến fault.
 - Không chắc là fault sẽ luôn luôn gây ra failure !
- ❖ **Defect**: là khiếm khuyết của phần mềm so với mong đợi của người sử dụng.
 - Phần mềm không lỗi nhưng vẫn có defect
- ❖ **Bug**: là hư hỏng trong hệ thống, có thể là phần cứng hoặc phần mềm.

1. Phần mềm có chất lượng là phần mềm không còn lỗi
=> **tìm và sửa hết lỗi (error) ?**
 - Phần mềm không còn lỗi chưa chắc đã có chất lượng.
 - Dijkstra: kiểm thử chỉ khẳng định PM còn lỗi, không thể khẳng định PM hết lỗi (*“SW Testing & QA from theory to practice”, P13*).
2. Phát hiện và sửa lỗi trên các ấn phẩm ban đầu (đặc tả yêu cầu, thiết kế) là rất cần thiết, để **tránh phát tán lỗi sang các công đoạn theo sau**.
 - Làm đúng theo yêu cầu sai : phải làm lại !
3. Tự động hoá việc kiểm thử là rất cần thiết.
 - Có lỗi → sửa → kiểm thử lại toàn bộ sản phẩm.

- ❖ Là hành động tìm chỗ không phù hợp của ấn phẩm PM so với yêu cầu của công đoạn/mong đợi đối với ấn phẩm, để chỉ ra rằng PM **chưa thỏa mãn** yêu cầu (errors)/mong đợi (defects) **trong các trường hợp đã kiểm thử** (testcase).
 - Testcase là một trường hợp kiểm thử để biết một ấn phẩm/phiên bản của PM có bị lỗi không.
- ❖ Mục đích của kiểm thử là tìm error/defect, để sửa – cải tiến chất lượng => kiểm thử không chỉ khẳng định PM còn lỗi; nó còn phải **đưa ra chứng cứ đầy đủ về lỗi** để tìm nguyên nhân và cách khắc phục.
- ❖ Một lỗi được phát hiện có thể **không cần phải sửa ngay**, vì nhiều lý do:
 - Dành thời gian để sửa lỗi nghiêm trọng hơn
 - Trì hoãn, để sửa chung với các lỗi khác có liên quan.
 - Thay đổi cách dùng PM để tránh cho nó đụng đến lỗi.

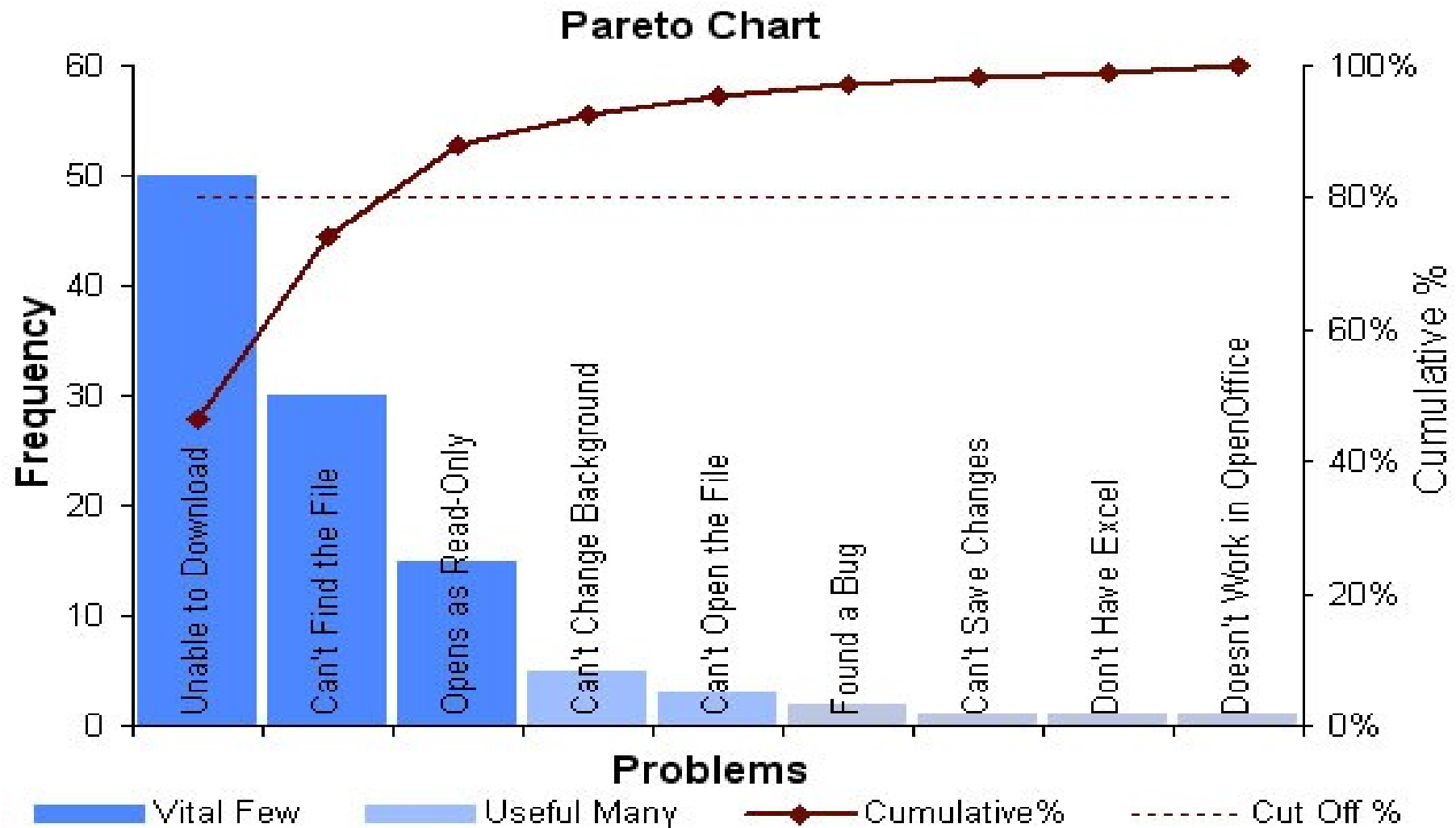
Chiến lược kiểm thử (test-strategies)

7

- ❖ Là cách tiếp cận để xây dựng kế hoạch kiểm thử có hiệu quả, nhằm giảm bớt khối lượng kiểm thử (rất nhiều và ngày càng có nhiều trường hợp cần kiểm thử).
- ❖ Tài liệu này giới thiệu một số chiến lược kiểm thử, đó là:
 - a. Dùng nguyên lý Pareto, để kiểm thử đạt hiệu quả cao
 - b. Dùng “top-down” thay cho “bottom-up”, để kiểm thử sớm
 - c. Dùng gieo lỗi, để đánh giá năng lực kiểm thử

a. Nguyên lý Pareto (luật 20/80)

8



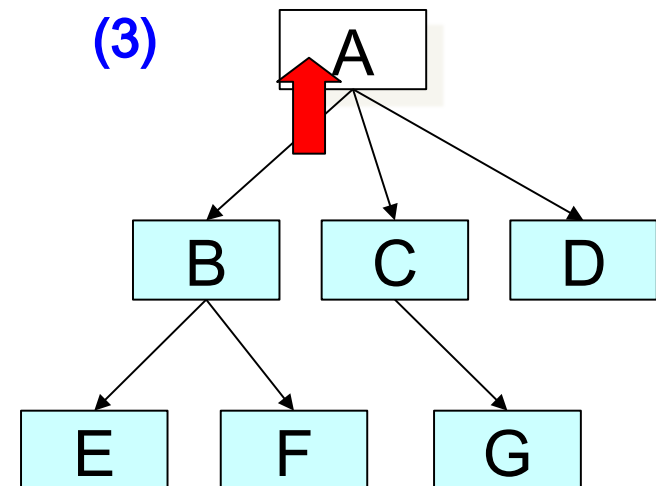
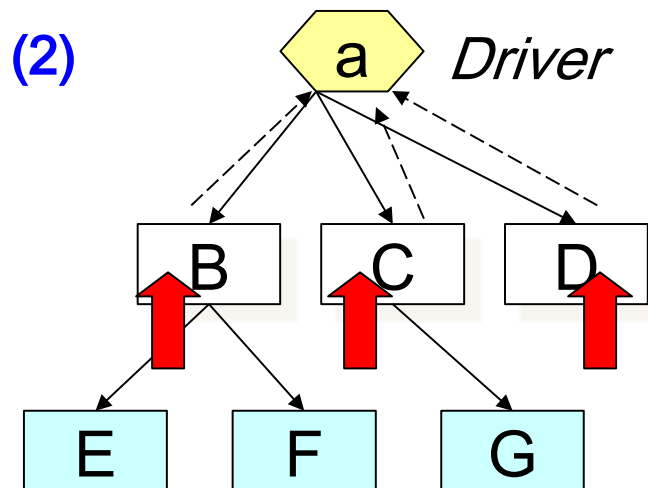
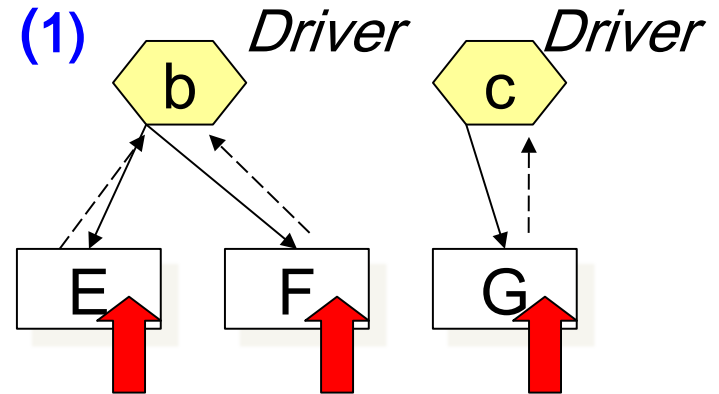
Mô đun (hoặc chức năng) có tần suất dùng nhiều, hoặc có thể gây sự cố nghiêm trọng được ưu tiên kiểm thử và sửa trước.

b. Bottom-up vs Top-down (1/3)

9

Driver: là một mô-đun “rỗng” (b, c) dùng để tạo ra các lệnh gọi tới các mô-đun mức thấp (E, F, G) để kiểm tra các mô-đun mức thấp có hoạt động đúng không. Các mô-đun chương trình nằm ở mức thấp nhất được kiểm tra trước, rồi đến các mô-đun ở trên, là các mô-đun có lời gọi đến các mô-đun mức dưới.

Bottom-up :

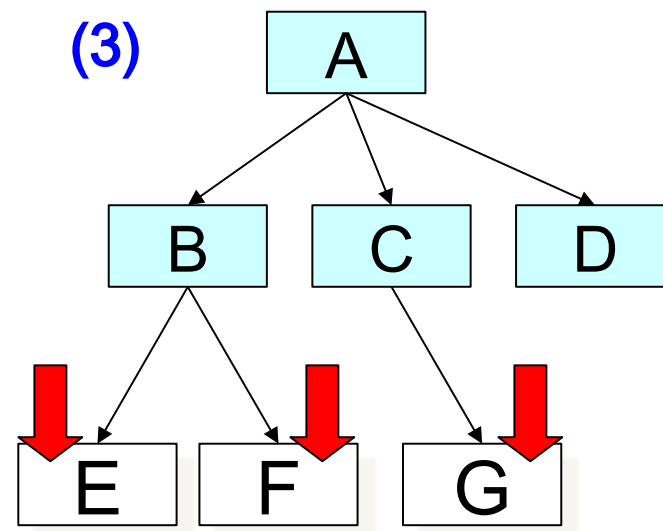
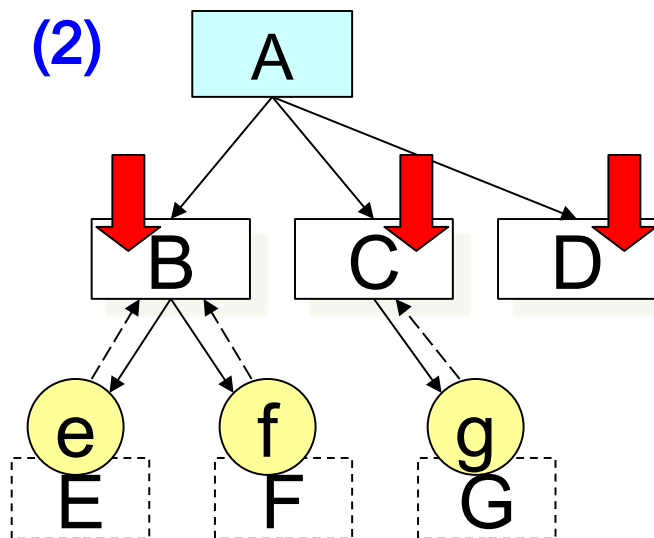
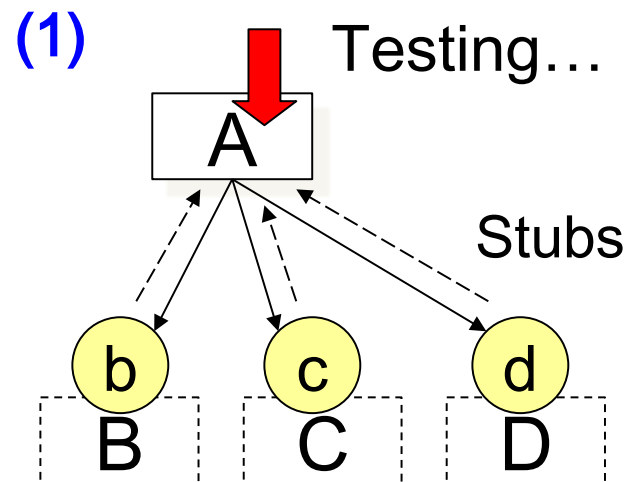


Bottom-up vs Top-down (2/3)

10

Stubs: là các mô-đun “rỗng” (b, c, d) thay thế cho mô-đun thật (B, C, D) để xem mô-đun mức cao (A) có hoạt động đúng không. Các stubs chỉ được cài đặt các giao tiếp (inputs, outputs) và các bảng chuyển đổi inputs-outputs tính sẵn cho các testcases chứ không được cài đặt code chính thức.

Top-down:



Bottom-up vs Top-down (3/3)

11

- ❖ Top-Down: các mô-đun chính được viết và test trước (đối chiếu với chức năng) - như vậy tính logic được nâng cao khi các mô-đun được kiểm thử xong.
- ❖ Bottom-Up: không có chương trình chạy được cho tới khi mô-đun cuối cùng (mô-đun cấp cao, system) được test và tích hợp – lúc này các lỗi thiết kế mới có thể phát hiện và có thể phải làm lại các mô-đun con đã pass qua kiểm thử !

Như vậy việc sử dụng Top down thay cho Bottom up sẽ hạn chế việc sửa lại các mô-đun đã được làm đúng theo yêu cầu sai.

c. Gieo lồi (1/2)

12

- ❖ Gieo lồi để ước tính số lồi còn sót lại trong phần mềm, tương tự như việc đếm (ước tính) số cá trong ao:
1. Bắt **N** con cá trong ao nuôi cá.
 2. Đánh dấu hết N con cá này và thả lại vào ao.
 3. Quăng 1 mẻ lưới bắt được **M** con; trong đó có **n** con có đánh dấu.
 4. Gọi **X** là tổng số cá trong ao, dựa trên giả định rằng các con cá có đánh dấu và không được đánh dấu hoàn toàn trộn lẫn vào nhau, thì ta có tỉ lệ này:

$$\frac{M}{X} = \frac{n}{N} = \frac{M - n}{X - N}$$

❖ Giả sử:

- Số lỗi được gieo vào PM là **N (total seeds)**
- Số lỗi phát hiện được là M, trong đó có **n** lỗi được gieo (**detected seeds**)
- Vậy có **M-n** lỗi được phát hiện nhưng không được gieo (detected non seeds).
- Gọi số lỗi của PM sau khi gieo lỗi là **X**, thì số lỗi tự có của PM là **X-N** (total non seeds).

❖ Hệ số phát hiện lỗi C_t (sau một thời gian t) của kiểm thử là

$$C_t = \frac{\text{detected seeds } (n)}{\text{total seeds } (N)} = \frac{\text{detected non seeds } (M-n)}{\text{total non seeds } (X-N)}$$

Thiết kế testcases (test-design)

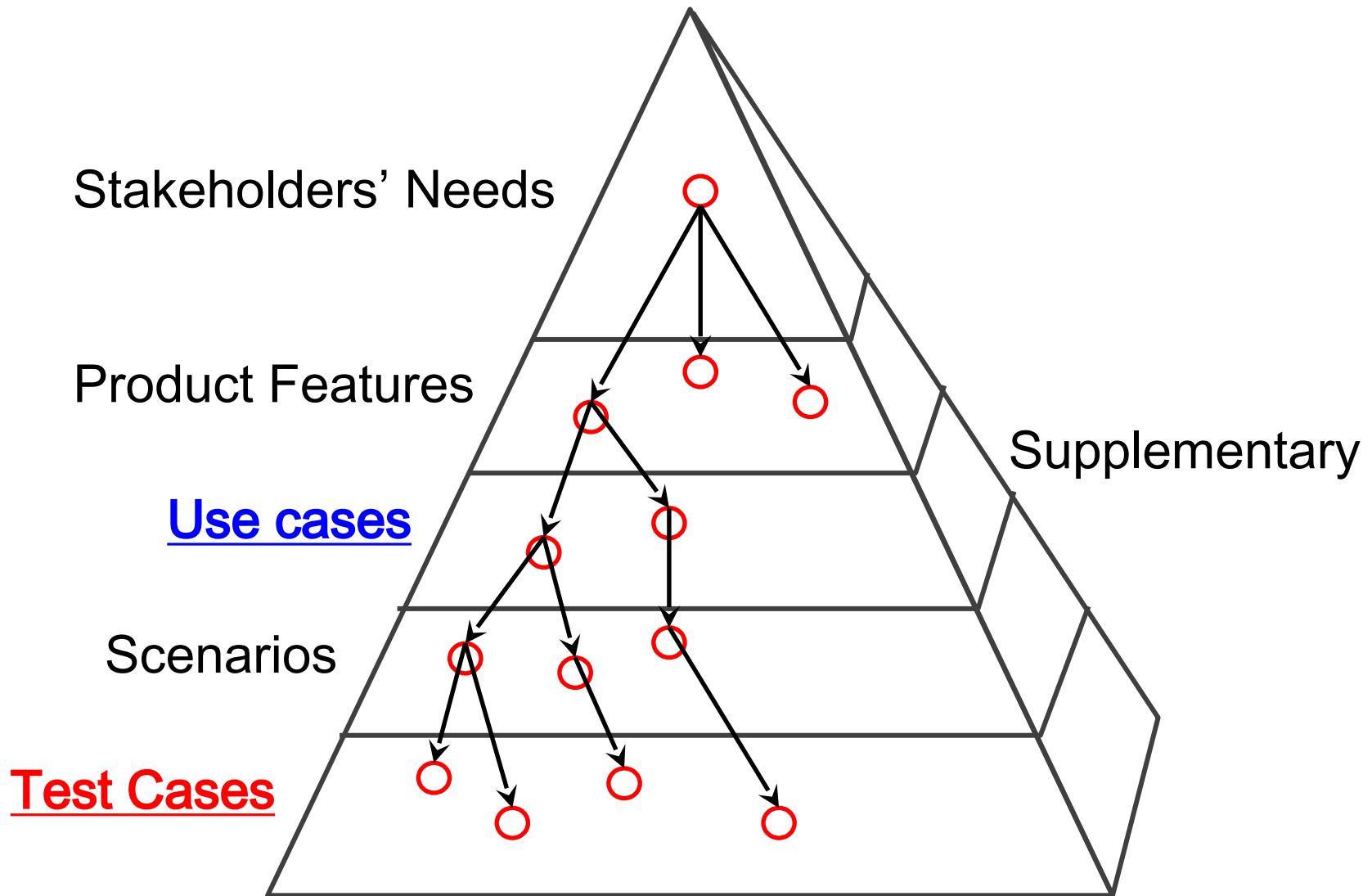
14

What subset of all possible testcases has the highest probability of detecting the most errors ?

- a. Thiết kế testcase từ usecase
- b. Thiết kế testcase trong black box test
- c. Thiết kế testcase trong white box test

a) Thiết kế testcase từ usecase

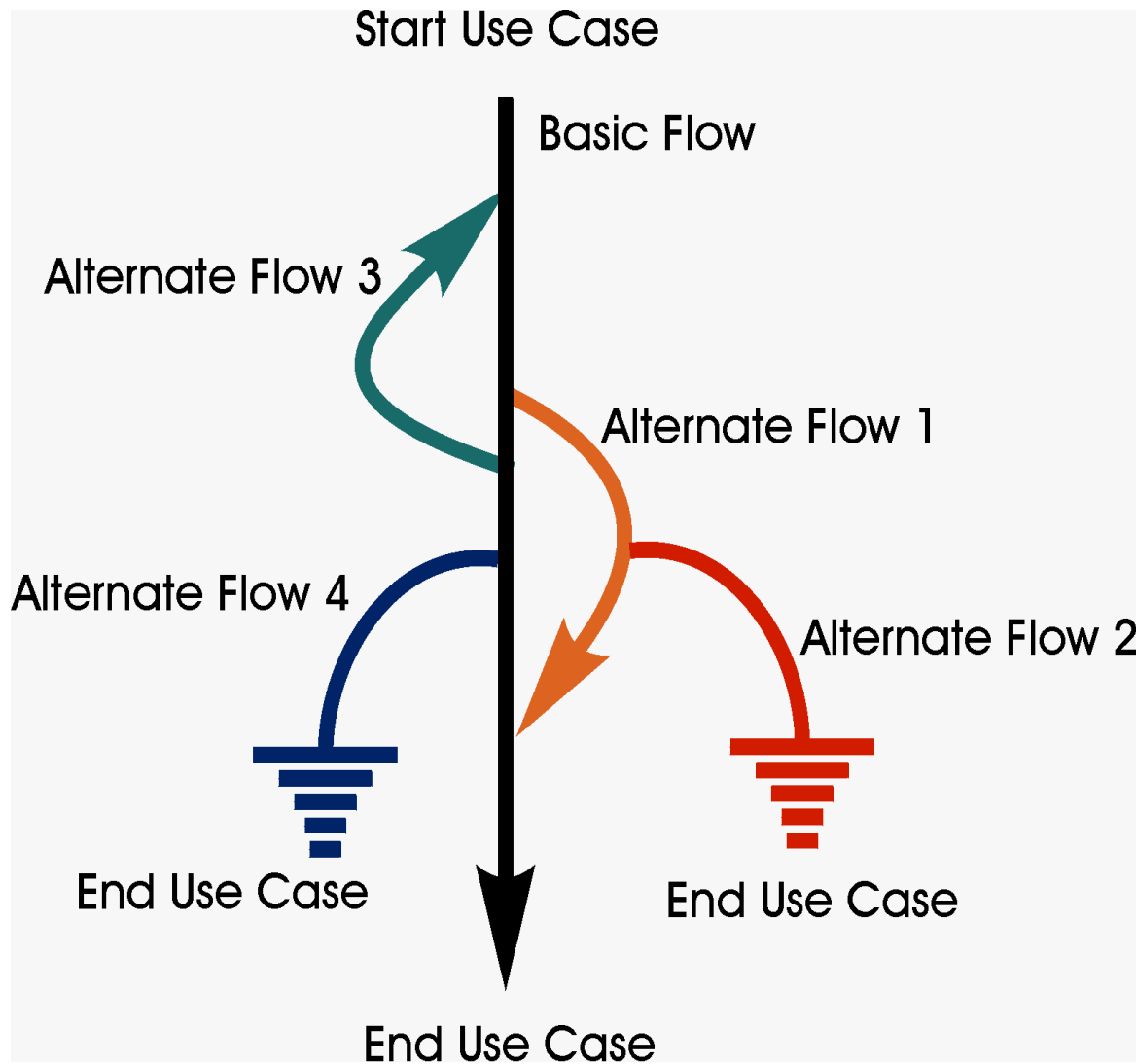
15



Testcase được phát sinh rất sớm, để trợ giúp kiểm thử các ấn phẩm trong lúc chúng đang được làm ra.

Lợi ích:

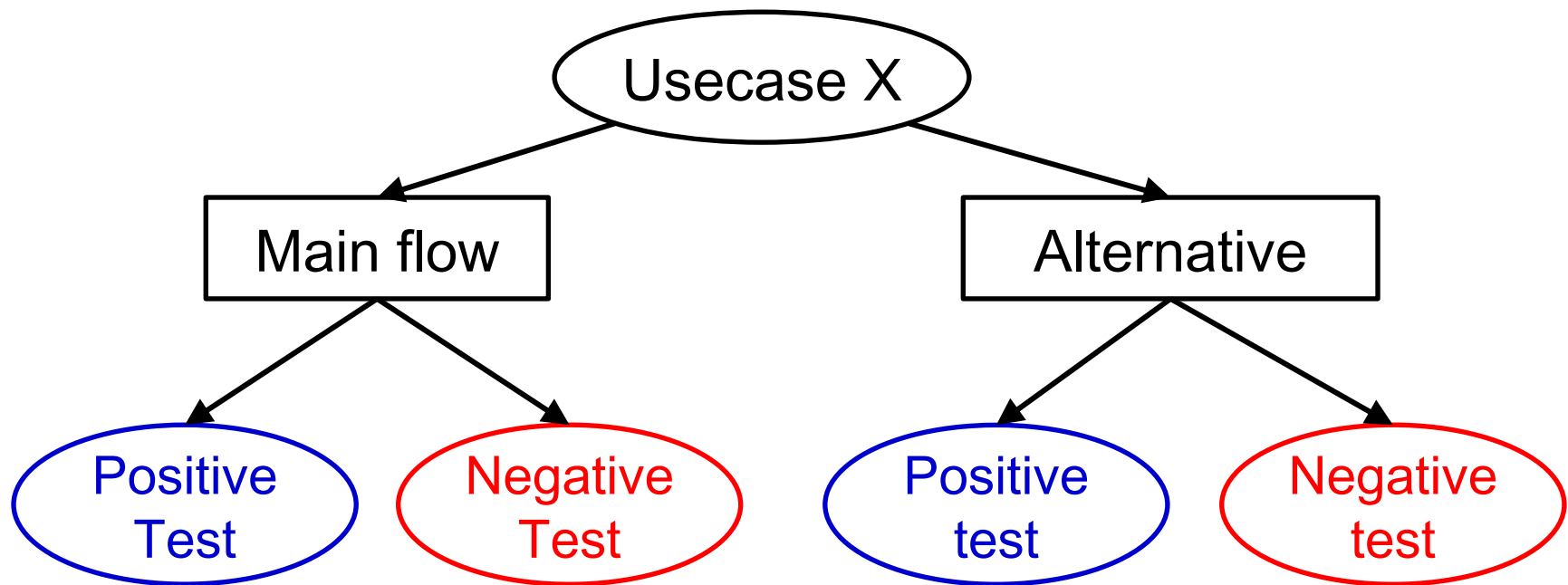
- Có thể test thủ công trên ấn phẩm thiết kế.
- Có thể test ngay source code (bằng cunit, nunit,...)
- Sửa lỗi ngay khi tiếp cận với top-down
- Phù hợp với acceptance test



Usecase diễn tả tương tác giữa PM với các actors tùy theo tình huống sử dụng PM.

Diễn biến của tương tác thể hiện trong dòng xử lý chính (main flow) và các ngoại lệ (alternative flows)

Mỗi dòng xử lý là một Scenario/flow trong lược đồ usecase



- ❖ Testcases được phát triển từ main-flow & alternatives
 - Positive testcase : valid input → expected “kết quả”
 - Negative testcase : invalid input → expected “từ chối” (input không nằm trong miền giá trị hợp lệ của usecase)
- ❖ Dữ liệu test dựa trên thông điệp của các flows.
 - 1 bộ input có thể tạo ra nhiều expected (testcases phải bao phủ các expected này).

- ❖ Testcase được tạo ra từ các tiêu chí kiểm thử phần mềm:
 - PM phải là công cụ hữu ích trong usecase của nó
- ❖ Các tiêu chí kiểm thử được xếp vào 3 mức : concepts, design và implementation (giống như thiết kế CSDL)
- ❖ Testcase được thực hiện thủ công hoặc tự động.
 - Testing Automation, và AI có thể trợ giúp => Có thể tự động hóa các testcase đang được làm thủ công hay không ?

b) Thiết kế testcase trong black box test 20

Là các kỹ thuật kiểm thử PM bằng cách kiểm tra đối chiếu kết quả xử lý của nó so với yêu cầu / mong đợi mà không cần biết cấu trúc bên trong của nó.

Các công đoạn làm phần mềm sử dụng Black Box Testing:

- ❖ **Kiểm thử chức năng** (Functional Testing): đảm bảo phần mềm thực hiện đúng các chức năng được yêu cầu
- ❖ **Kiểm thử hệ thống** (System Testing): Đánh giá phần mềm có đáp ứng đầy đủ các yêu cầu kỹ thuật và nghiệp vụ không (Bao gồm kiểm thử hiệu năng, bảo mật, khả năng tương thích, và tính ổn định)
- ❖ **Dùng cho validation**: Tiêu chí kiểm thử (và testcases) được đưa ra từ mong muốn về phần mềm

❖ Ưu điểm trong SQA

- Tự động hóa được với PM thực thi
- Dùng lại các testcase cũ (cho regression test)
- Tester không cần biết mã nguồn (chỉ cần input/output)
- Bám sát mong đợi từ môi trường (phát hiện defects)
- Thực hiện được bởi bên thứ ba độc lập

❖ Nhược điểm trong SQA

- Các testcase thường bị trùng lặp (→ cần white box)
- Không phát hiện được xử lý dở, hoặc nguy hiểm (lỗi hồng bảo mật)

1. Decision technique

Lập bảng quyết định nếu miền giá trị của input và output là rời rạc hữu hạn

Ví dụ: tiêu chí kiểm thử : check email với password

- ❖ Email : exists, not exist
- ❖ Password : matched, ***

	Testcase-1	Testcase-2	Testcase-3
Email	Exists	Exists	Not exist
Password	Matched	Not match	***
Expected	(logon)	"incorrect password"	"incorrect email"

POS.Test

POS.Test

NEG.Test

2. Equivalent Class Partitioning

Phân hoạch miền giá trị inputs thành các miền giá trị tương đương: mọi giá trị trong một miền được phần mềm ứng xử như nhau (1 flow của usecase) => chỉ cần chọn một giá trị đại diện trong một miền để kiểm thử.

- Ví dụ: nếu mua từ 4 đến 10 bánh pizza thì từ cái thứ 4 giá giảm 50%; trên 10 bánh là không hợp lệ.

Số lượng	≤ 0	1-3	4-10	>10
Test value	0	1	5	11
Expected	"Invalid"	giá thường	giá giảm	"invalid"

NEG.Test

POS.Test

POS.Test

NEG.Test

- ❖ Giả sử PM có 6 inputs $I_1, \dots, I_6$ và 3 outputs O_1, O_2, O_3
 - Nếu mỗi input có 5 lớp tương đương thì để test cho 6 inputs cần thực hiện khoảng $5^6 = 15.625$ test cases !
- ❖ Giả sử:
 - O_1 chỉ phụ thuộc $I_1, I_2, I_3 \rightarrow \{I_1, I_2, I_3\}$ là ốc đảo của O_1
 - O_2 chỉ phụ thuộc $I_4, I_5 \rightarrow \{I_4, I_5\}$ là ốc đảo của O_2
 - O_3 chỉ phụ thuộc $I_6 \rightarrow \{I_6\}$ là ốc đảo của O_3
- ❖ Như vậy:
 - Đối với O_1 : chỉ cần test bộ I_1, I_2, I_3 : 5^3 test cases
 - Đối với O_2 : chỉ cần test bộ I_4, I_5 : 5^2 test cases
 - Đối với O_3 : chỉ test I_6 : 5 test cases
- ❖ Theo cách này, tổng số test cases cần thực hiện là $5^3 + 5^2 + 5 = 155$ (thay vì 15.626) test cases.

3. Boundary Value Analysis

- ❖ Phân tích giá trị biên: Tập trung kiểm thử các giá trị ngay sát ranh giới của điều kiện (min, max, ngay trên/dưới giới hạn).
 - Vì lỗi thường xảy ra ở vùng cận biên.
- ❖ Ví dụ: Tuổi hợp lệ là từ 18 đến 60
 - Kiểm thử các giá trị 17 và 61 (bên ngoài biên)
 - Kiểm thử các giá trị 18 và 60 (bên trong biên)

4. State Transition Testing

26

- ❖ Kỹ thuật kiểm thử trạng thái: dùng để kiểm tra hành vi của hệ thống khi trạng thái thay đổi theo chuỗi sự kiện (theo mô hình chuyển trạng thái - State Transition Diagram).
- ❖ Khi nào nên dùng kỹ thuật này?
 - Khi hệ thống có logic trạng thái rõ ràng.
 - Khi hành vi phụ thuộc vào lịch sử tương tác trước đó.
 - Khi cần kiểm tra hành vi tương ứng với sự kiện
- ❖ Ví dụ: mở khóa cửa bằng số pin
 - Nhập lần 1: pin đúng → mở cửa, pin sai → cho nhập lại
 - Nhập lần 2: pin đúng → mở cửa, pin sai → cho nhập lại
 - Nhập lần 3: pin đúng → mở cửa, pin sai → cho nhập lại
 - Nhập lần 4: pin đúng → mở cửa, pin sai → cho nhập lại
 - Nhập lần 5: pin đúng → mở cửa, pin sai → báo trộm

5. Cause & Effect Graph

27

Là kỹ thuật kiểm thử chức năng, bằng cách diễn tả yêu cầu ban đầu thành điều kiện ràng buộc giữa input -“nguyên nhân” và output-”hậu quả” của phần mềm để đưa ra các testcases

Trình tự xác định testcase gồm các bước:

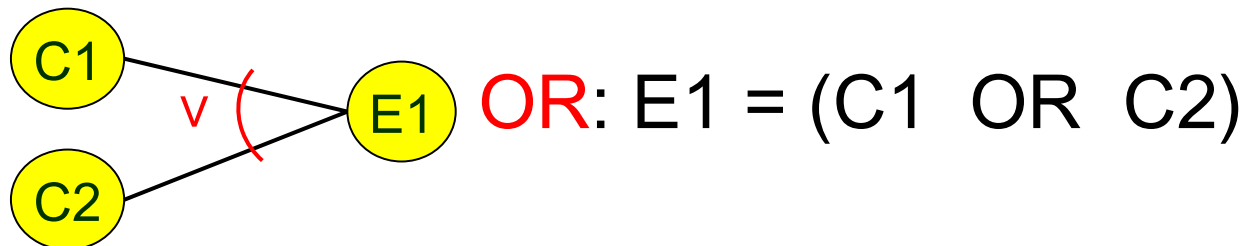
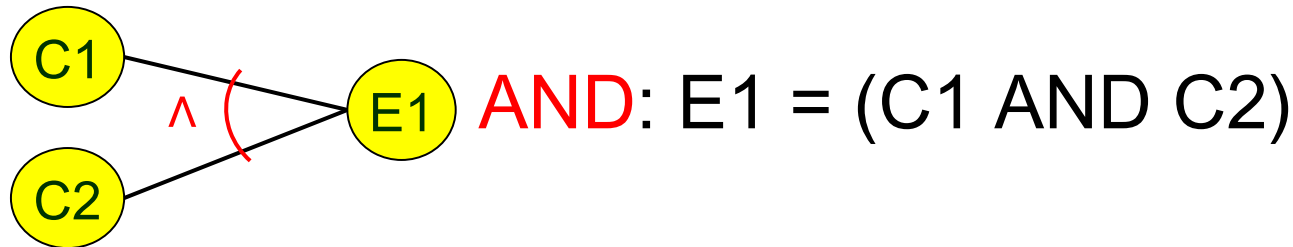
1. Phân tích các trạng thái logic của yêu cầu
2. Từ yêu cầu, vẽ lược đồ cause-effect
3. Từ lược đồ, lập bảng quyết định tình huống
4. Từ bảng, tạo các testcase

<https://www.softwaretestinghelp.com/cause-and-effect-graph-test-case-writing-technique/>

Cause & Effect Graph: ký hiệu

28

- ❖ Nếu tình huống input (i) xuất hiện thì $C_i = \text{True}$
- ❖ Nếu kết xuất (j) xuất hiện thì $E_j = \text{True}$
- ❖ Các toán tử luận lý AND, OR, NOT:



Yêu cầu nghiệp vụ: Chương trình cập nhật file và in thông điệp tùy theo 2 ký tự nhập vào:

Causes:

- Ký tự đầu: phải là A hoặc B
- Ký tự kế tiếp: là số, vd: A2

Effects:

- Nếu 2 ký tự nhập vào là hợp lệ, cập nhật file
- Nếu ký tự đầu không hợp lệ ($\neq A B$), in thông điệp X
- Nếu ký tự kế không phải là số, in thông điệp Y

❖ Định nghĩa các C_i (“nguyên nhân”):

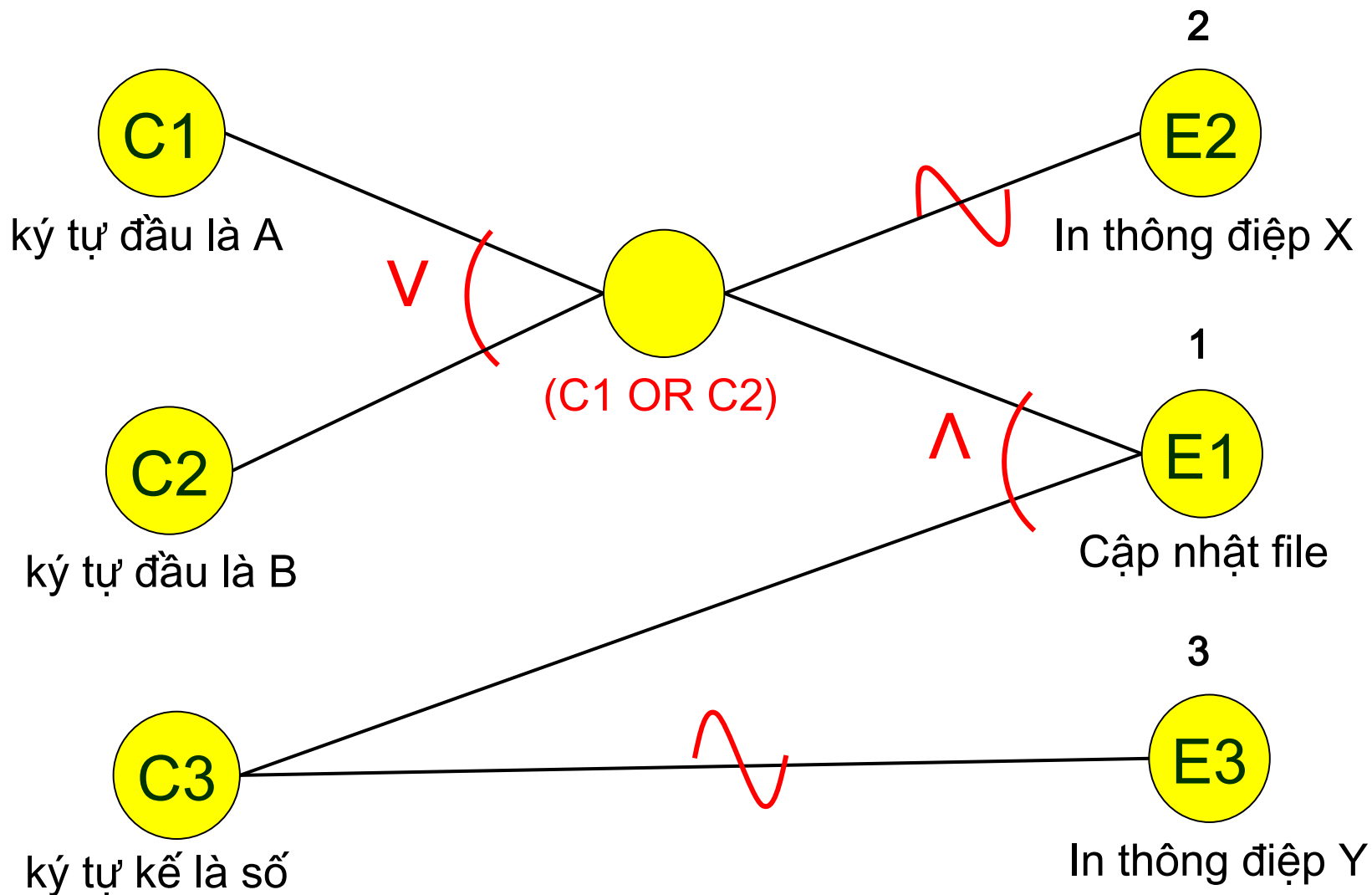
- C_1 = ký tự đầu là A
- C_2 = ký tự đầu là B
- C_3 = ký tự kế là ký số 0..9

❖ Định nghĩa các E_j (“hậu quả”)

- E_1 = Cập nhật file (positive test)
- E_2 = In thông điệp X (negative test)
- E_3 = in thông điệp Y (negative test)

Bước 2: vẽ lược đồ cause-effect

31



Các tiêu chí kiểm thử (mức ý niệm)

Bước 3: lập bảng quyết định

32

1	2		3	4
Case	TC1	TC2	TC3	TC4
C1	T		F	
C2		T	F	
C3	T	T		F
E1	T	T		
E2			T	
E3				T



Các tiêu chí kiểm thử (mức thiết kế)

Bước 4: thiết lập testcases

33

ID	Name	Des.	Steps	Expected
TC1	UpdFile-1	...	1.Enter 1rst char = 'A' 2.Enter 2 nd char = '5'	File update
TC2	UpdFile-2	...	1.Enter 1rst char = 'B' 2.Enter 2 nd char = '6'	File update
TC3	Print-X	...	1.Enter 1rst char = 'C' 2.Enter 2 nd char = '6'	Print X
TC4	Print-Y	...	1.Enter 1rst char = 'A' 2.Enter 2 nd char = 'C'	Printf Y

Các tiêu chí kiểm thử (mức hiện thực - testcase)

c) Thiết kế testcase trong white box test 34

Là các kỹ thuật kiểm thử giải pháp của PM bằng cách nhìn vào cấu trúc luận lý bên trong của nó (thiết kế, mã lệnh) để kiểm chứng cách biến đổi inputs thành outputs theo yêu cầu trên ấn phẩm (bản thiết kế, source code).

Ví dụ, các trường hợp áp dụng White-box Testing:

- ❖ **Unit Testing** (Kiểm thử đơn vị): Kiểm tra từng hàm, phương thức hoặc module nhỏ nhất của phần mềm đảm bảo từng đơn vị hoạt động đúng như thiết kế
- ❖ **Integration Testing** (Kiểm thử tích hợp): Kiểm tra luồng dữ liệu và logic giữa các môđun khi tích hợp các module
- ❖ **Hỗ trợ Black-box test**: White box phân tích các hành vi xử lý bên trong phần mềm để đưa ra các testcase để thực thi tự động bằng Black-box test

- ❖ Ưu điểm của whitebox test trong SQA
 - Giúp phát hiện lỗi logic, lỗi bảo mật, hoặc đoạn mã không được thực thi
 - Bắt đầu sớm trong tiến trình làm PM (không cần code thực thi).
 - Giúp định nghĩa hoặc giới hạn các testcase cho Blackbox testing

- ❖ Khuyết điểm trong SQA
 - Không dùng lại được sau khi sửa lỗi.

Testing Coverage (độ phủ của testcase) 36

Số lượng testcase thực hiện được là hữu hạn; trong khi đó số trường hợp xử lý của phần mềm có thể rất lớn (số mã lệnh * trạng thái dữ liệu), không khả thi cho việc tạo testcase một cách toàn diện (không bỏ sót trường hợp nào). Vấn đề là phải có kỹ thuật khả thi để đưa ra tập testcase tương đối phủ hết các trường hợp xử lý của phần mềm.

Độ bao phủ của phương pháp test:

Test coverage = số trường hợp được kiểm thử / tổng số trường hợp mà phần mềm cần xử lý

Trong white box test, các testcase được xác định từ các trường hợp mà chương trình xử lý (các điều kiện rẽ nhánh: miền giá trị của biến điều kiện quyết định cách xử lý của chương trình)

```
function F(a, b)
{
    if (a > 5)           // Atomic boolean C1: a > 5
        S1;
    else
        S2;

    if ( (a % 2 == 0) || (b % 3 == 0) ) // C2 & C3
        S3;
    else
        S4;
}
```

IF(a>5) S1 else S2; IF(a%2==0) || (b%3==0) S3 else S4

1. Statement coverage: mỗi lệnh được thực thi ít nhất một lần.

- test 1 : (a = 6, b = *) --> S1, S3
- test 2 : (a = 1, b = 2) --> S2, S4

2. Branch coverage: mỗi exec-path thực thi ít nhất một lần.

- test 1 : (a = 8, b = *) --> S1 - S3
- test 2 : (a = 7, b = 4) --> S1 - S4
- test 3 : (a = 1, b = *) --> S2 - S3
- test 4 : (a = 3, b = 4) --> S2 - S4

IF(a>5) S1 else S2; IF(a%2==0) || (b%3==0) S3 else S4

3. Condition coverage: mỗi mệnh đề logic cơ bản trong điều kiện rẽ nhánh (atomic boolean) được ước lượng với 2 trường hợp :True, và False

- C1 = (a>5) có 2 trường hợp
- C2 = (a%2==0) có 2 trường hợp
- C3 = (b%3==0) có 2 trường hợp

Tổ hợp các trường hợp ta có $2 * 2 * 2 = 8$ testcases

- C1 & C2 & C3: (a>5) & (a%2==0) & (b%3==0)
- C1 & C2 & !C3: (a>5) & (a%2==0) & (b%3!=0)
- C1 & !C2 & C3: (a>5) & (a%2!=0) & (b%3==0)
- C1 & !C2 & !C3: (a>5) & (a%2!=0) & (b%3!=0)
- !C1 & C2 & C3: (a≤5) & (a%2==0) & (b%3==0)
- !C1 & C2 & !C3: (a≤5) & (a%2==0) & (b%3!=0)
- ...

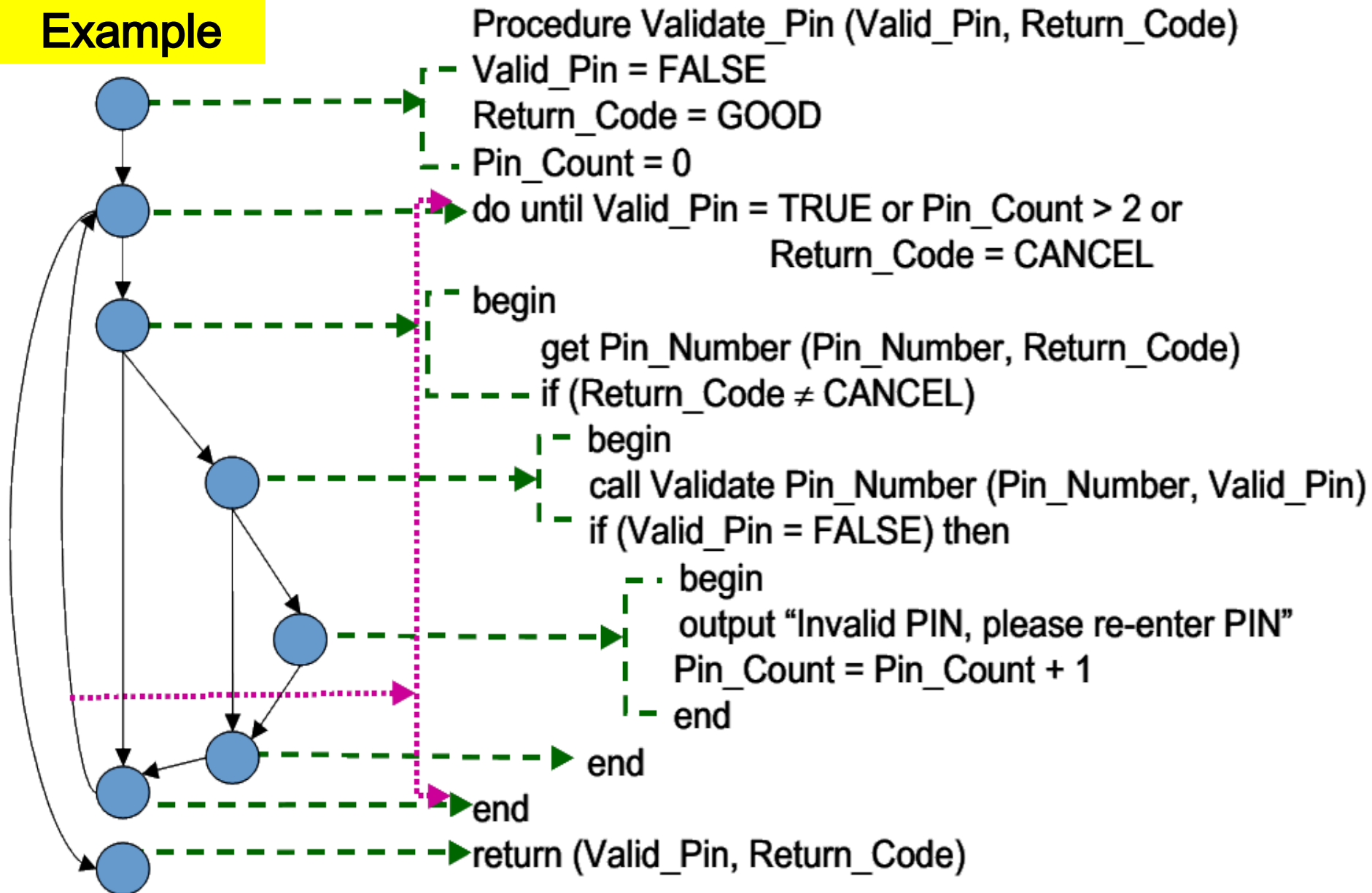
Nếu một chương trình đã được test và sửa hết lỗi thì có phải là nó không còn lỗi, hay bộ testcase chưa đủ để phát hiện các lỗi còn lại ?

Nguyên lý McCabe (Thomas J. McCabe, 1976): chỉ ra số test-case **tối thiểu** cho một function

- ❖ Function được phân tích các trường hợp thực thi (1 cách chạy, là 1 control-flow, hoặc 1 control path)
- ❖ Đếm số control-flow để tính số testcase tối thiểu: **Mỗi control-flow phải có ít nhất 1 testcase.**

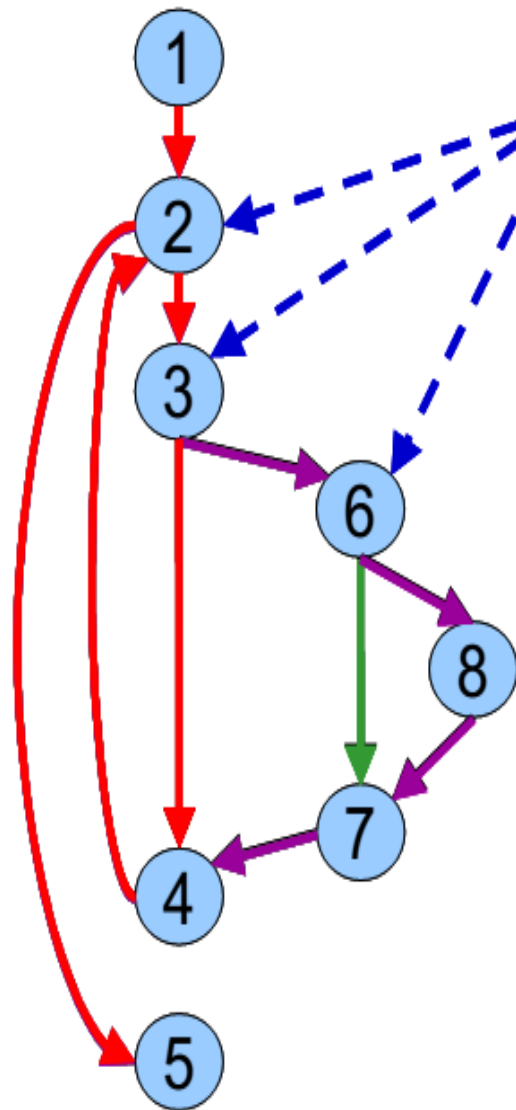
Control Flow Graph (CFG)

Example



CFG: Basis paths from example

42



Predicate Nodes

Path 1: 1,2,5

Path 2: 1,2,3,4,2,5

Path 3: 1,2,3,6,7,4,2,5

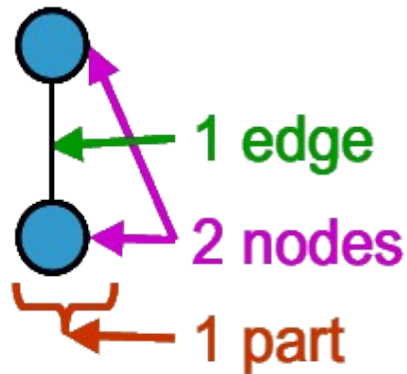
Path 4: 1,2,3,6,8,7,4,2,5

Cyclomatic complexity (McCabe, 1976)

43

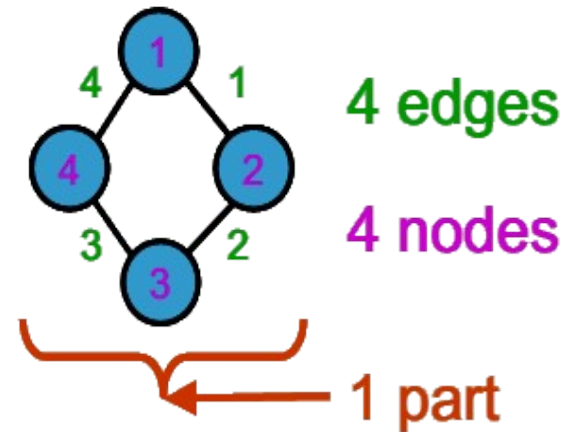
Model: $V(G) = \text{edges} - \text{nodes} + 2p$

where p = number of unconnected parts of the graph



$$V(G) = 1 - 2 + 2 \times 1 = 1$$

Straight line code always
has a complexity of 1



$$V(G) = 4 - 4 + 2 \times 1 = 2$$

McCabe: The minimum number of test-cases is $V(G)$